

ENTREGA TRABAJO PRÁCTICO N°III - PROYECTO ITERA

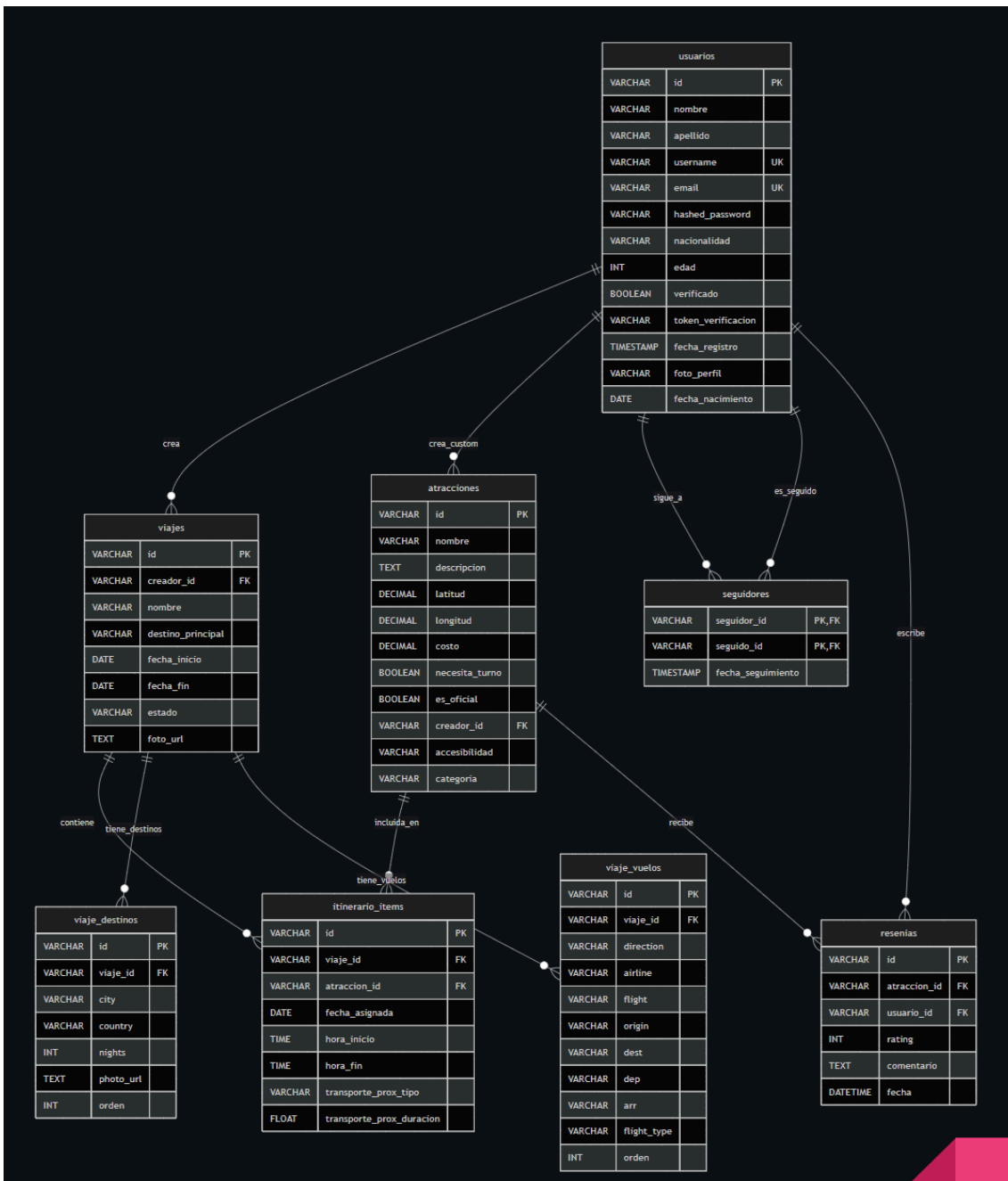
Laboratorio de Desarrollo de Software

Grupo 10: Astorga Lucia, Borodij Candela y Militello Santiago

Link al repositorio de GitHub:

<https://github.com/UCH-LDS-2026/grupo-10/tree/main/database>

Diagrama de Entidad-Relación:



Arquitectura del Sistema: Itera (Plataforma de Planificación de Viajes)

El sistema Itera está diseñado utilizando una Arquitectura Cliente-Servidor estructurada en capas, enfocada en la separación de responsabilidades para garantizar la escalabilidad, seguridad y facilidad de mantenimiento. La comunicación entre el cliente y el servidor se realiza a través de una API RESTful.

Componentes principales de la arquitectura:

1. Frontend (Capa de Presentación / Cliente)

Tecnologías: JavaScript, HTML5 y CSS3 (integrando Node.js para la gestión de dependencias del entorno de desarrollo).

Descripción: Esta capa es responsable de la interfaz de usuario (UI) y la experiencia de usuario (UX). Al utilizar JavaScript para el manejo del DOM, se logra una aplicación interactiva (Single Page Application o similar) que permite a los usuarios armar itinerarios de viaje, visualizar mapas y gestionar destinos de forma fluida.

Comunicación: El Frontend consume los endpoints de la API expuesta por el Backend mediante peticiones HTTP asíncronas (AJAX/Fetch), procesando respuestas en formato JSON.

2. Backend (Capa de Lógica de Negocio y Servicios)

Tecnologías: Java 17+ y el framework Spring Boot.

Patrón Arquitectónico Interno: Se utiliza una arquitectura multicapa típica de Spring Boot:

Controladores (Controllers): Exponen los endpoints REST. Reciben las peticiones HTTP del frontend, validan los datos de entrada y delegan el procesamiento a la capa de servicios.

Servicios (Services): Contienen toda la lógica de negocio del sistema (ej. validar disponibilidad de fechas, cálculos de itinerarios, gestión de usuarios).

Repositorios (Repositories - Spring Data JPA): Actúan como la capa de acceso a datos, abstrayendo las consultas SQL y gestionando el mapeo objeto-relacional (ORM) con las entidades de la base de datos.

3. Base de Datos (Capa de Persistencia)

Tecnologías: MySQL 8.x Se utiliza una base de datos relacional para garantizar la integridad de los datos y el cumplimiento de las propiedades ACID en las transacciones. Dado que el dominio del sistema tiene una estruct.

Descripción: ura altamente interconectada (un Usuario tiene Viajes, un viaje tiene Destinos y Vuelos, y un destino incluye un Itinerario con Atracciones), el modelo relacional es la opción más sólida.

4. Integraciones de Terceros (Sistemas Externos)

APIs Externas: El sistema se apoya en servicios externos (por ejemplo, Google Places API) para nutrir de contenido dinámico la aplicación.

Función: Cuando un usuario realiza una búsqueda de "Atracciones", el backend o el frontend se comunican con estas APIs externas para obtener información en tiempo real, metadatos, valoraciones e imágenes de los lugares turísticos, enriqueciendo así la base de datos propia.

Link para ver archivo schema.sql:

<https://github.com/UCH-LDS-2026/grupo-10/blob/main/database/schema.sql>